

Jeffrey Taylor

CPET 461 - Real Time Operating Systems - Lab 7 Report

Abstract

The objective of this lab was to create a FreeRTOS application to perform a fixed amount of work using a varying number of tasks. The work in question was to calculate if 65530 is a “Perfect Number”, meaning 65530 a positive number that is the sum of its divisors. This calculation will be done a total of 180 times, using 1, 2, 3, 4, 6, and 10 tasks. Depending on the number of tasks, the calculation will be performed by each task $180 / N$ number of tasks times. For a total of 180 total.

Experimental Setup

The project was implemented using FreeRTOS configured through STM32CubeMX. A worker-task model was used where a variable number of tasks were created depending on the test case. The number of tasks used in the experiment were 1, 2, 3, 4, 6, and 10. Each worker task performed a total of 180 divided by the number of tasks, so that the sum of all work is 180. A helper function was written to determine whether the number 65530 is a perfect number. Each worker task repeatedly called this function for its assigned number of iterations. No delays were used in the worker tasks so that execution time would reflect only the computational workload, with potential error as each worker calls `Print_Line` to output what worker has started working. A shared global counter was used to track the number of completed tasks. Once a worker task finished its assigned iterations, it incremented this counter and terminated itself. A default task

continuously monitored this counter and determined when all worker tasks had completed execution.

Execution time was measured using the FreeRTOS function `xTaskGetTickCount()`. The start time was recorded before the worker tasks began execution, assumed tick 0, and the end time was recorded once all tasks had finished. The total execution time was calculated as the difference between these two values and was output to a serial terminal.

Results

Execution Time vs Number of Tasks

1 Worker: 2241 ticks

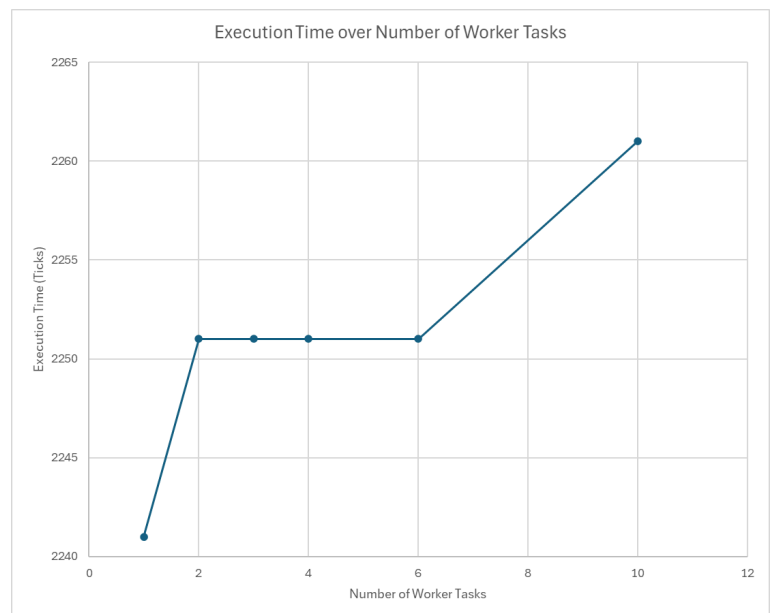
2 Workers: 2251 ticks

3 Workers: 2251 ticks

4 Workers: 2251 ticks

6 Workers: 2251 ticks

10 Workers: 2261 ticks



The results show that the execution time remained nearly constant as the number of worker tasks increased. Increasing slightly due to scheduler overhead.

Discussion

The results show that increasing the number of worker tasks did not significantly change the execution time. Across all configurations, the total runtime remained nearly constant, with only a slight increase as the number of tasks increased. This behavior occurs because the total amount of work being performed is fixed at 180 perfect-number calculations, regardless of how many tasks are used.

Although the workload is divided among multiple tasks, the system is running on a single-core processor. This means that only one task can execute at any given time, and FreeRTOS must schedule the tasks using time-slicing. As a result, increasing the number of tasks does not create true parallelism, but instead divides the same CPU time among more tasks.

The small increase in execution time at higher task counts, particularly with 10 tasks, can be attributed to scheduler overhead. As more tasks are created, the operating system must perform additional context switching and task management, which slightly increases the total runtime. Additionally, each worker task outputs a message using `Print_Line`, which introduces extra overhead and may contribute to minor timing variations, but we can consider this `Print_Line` negligible as each task calls the function.

Conclusion

This lab showed that increasing the number of FreeRTOS tasks does not significantly improve execution time for a fixed workload on a single-core system. The runtime remained nearly constant because tasks are time-shared rather than executed in parallel. A slight increase in execution time at higher task counts is due to scheduling overhead and additional task management.